

Exposé: Hauptprojekt / Masterarbeit

Behavioral Capability Contracts für MCP-basierte LLM-Agenten

Julius Hoffmann

Hochschule für angewandte Wissenschaften Hamburg
Arbeitsgruppe TeSSA: Sicherheit und Zuverlässigkeit
Betreuerin: Prof. Dr. Bettina Buth

1 Ausgangslage und Motivation

LLM-Agenten werden zunehmend nicht nur als Chat-Systeme, sondern als handlungsfähige Softwarekomponenten eingesetzt. Über Tool-Calling-Schnittstellen können sie Dateien lesen und schreiben, Netzwerkdienste ansprechen, Shell-Kommandos ausführen oder Anwendungen steuern. Das Model Context Protocol (MCP) standardisiert diese Verbindung zwischen Host, Client und Tool-Server. Dadurch können Agenten Werkzeuge dynamisch entdecken und auf Basis von Werkzeugnamen, Beschreibungen und Schemas auswählen.

Diese Flexibilität verschiebt die Sicherheitsgrenze. In klassischer Softwareanalyse steht häufig Code im Vordergrund. Bei MCP-basierten Agenten werden jedoch auch natürlichsprachliche Beschreibungen, JSON-Schemas und Tool-Ausgaben zu sicherheitsrelevanten Steuerinformationen. Ein Tool kann in seiner Beschreibung harmlos erscheinen, aber im Code weitergehende Fähigkeiten besitzen; eine Tool-Ausgabe kann indirekte Instruktionen enthalten; oder ein zunächst legitimes Tool kann nachträglich sein Verhalten ändern. Aktuelle Arbeiten zu MCP Security zeigen deshalb Angriffe wie Tool Poisoning, Tool Squatting, Rug-Pulls, überprivilegierte Tools und indirekte Prompt Injection [1–3, 5].

Die zentrale Motivation dieses Projekts ist, diese Lücke nicht nur als Erkennungsproblem, sondern als Contract- und Enforcement-Problem zu betrachten: Ein Agent sollte ein Tool nicht nur aufgrund dessen Beschreibung verwenden, sondern anhand einer expliziten, überprüfbaren Grenze dessen, was dieses Tool tun darf.

2 Problemstellung

MCP-Tools beschreiben ihre Funktion über Metadaten und Schemas. Diese Deklaration ist für den Agenten handlungsleitend, aber sie ist nicht automatisch vertrauenswürdig. Daraus ergeben sich drei Kernprobleme:

1. **Deklaration und Verhalten können auseinanderfallen.** Ein Tool kann laut Beschreibung lesen, aber tatsächlich schreiben; lokal arbeiten, aber Netzwerkzugriffe ausführen; oder sensitive Daten an unerwartete Ziele senden. Erste große

Messungen deuten darauf hin, dass solche Beschreibung-Code-Inkonsistenzen ein praktisches Problem im MCP-Ökosystem sind [6].

2. **Preflight-Analyse und Live-Beobachtung haben unterschiedliche Reichweite.** Tool Poisoning in Beschreibungen, offensichtliche Secrets oder Code-Sinks sind bereits vor einem produktiven Agentenlauf erkennbar. Ob ein Server seine Tool-Liste nachträglich ändert, ob eine Tool-Ausgabe den Agenten zu weiteren Aufrufen lenkt oder ob ein legitimes Tool gegen die Nutzerintention eingesetzt wird, zeigt sich dagegen erst im laufenden MCP-Kanal [5].
3. **Generische Guardrails sind zu grob.** Ein Proxy oder Gateway kann Tool-Aufrufe loggen, Schemas validieren oder einfache Regeln anwenden. Ohne einen tool-spezifischen Contract bleibt aber unklar, welches Verhalten für ein konkretes Tool legitim ist [11, 12].

Das Projekt untersucht deshalb, welche Capability-Signale bereits statisch oder vor der eigentlichen Nutzung ableitbar sind, welche erst bei gestarteten oder instrumentiert ausgeführten MCP-Servern sichtbar werden und wie beide Perspektiven in einem Capability Contract zusammengeführt werden können, der später zur Laufzeit durchsetzbar ist.

3 Zielsetzung

Das Gesamtziel ist die Konzeption und prototypische Umsetzung eines zweistufigen Sicherheitsansatzes für MCP-basierte LLM-Agenten:

- **Hauptprojekt:** Entwicklung eines offline nutzbaren CLI-Scanners, der MCP-Server anbindet, deren angebotene Tools und Schemas ausliest, erwartete Tool-Fähigkeiten aus Metadaten ableitet, beobachtbare Fähigkeiten durch statische Analyse und begrenzte instrumentierte Ausführung erfasst und Abweichungen als Findings sowie als Capability Contract ausgibt. Zugleich soll das Hauptprojekt herausarbeiten, welche Angriffsklassen durch statische Preflight-Signale gut adressierbar sind und welche erst durch Live-Beobachtung eines gestarteten MCP-Servers sichtbar werden.
- **Masterarbeit:** Entwicklung eines Laufzeit-Proxys, der diesen Contract auf echten Tool-Aufrufen erzwingt, Drift erkennt und bei sensiblen Operationen semantische Konsistenz zwischen Nutzerintention und Tool-Aufruf prüft.

Das Hauptprojekt liefert damit ein eigenständiges Ergebnis und zugleich die Grundlage für die Masterarbeit. Der Contract ist die Schnittstelle zwischen beiden Phasen.

4 Forschungsfragen

Für das Hauptprojekt stehen folgende Fragen im Vordergrund:

1. Wie lassen sich aus MCP-Tool-Beschreibungen und JSON-Schemas erwartete Fähigkeiten wie Dateisystemzugriff, Netzwerkzugriff, Prozessausführung oder Secret-Verwendung ableiten?

2. Wie lassen sich beobachtbare Fähigkeiten eines MCP-Servers für einen ersten Prototyp durch Codeanalyse und durch begrenzte instrumentierte Ausführung erkennen?
3. Welche MCP-Angriffsvektoren hinterlassen bereits vor der Nutzung belastbare statische Signale, und welche benötigen Live-Informationen wie Tool-Aufruf, Nutzerintention, Tool-Ausgabe oder Drift über die Zeit?
4. Welche Formen von deklarierter-vs.-beobachteter Divergenz sind praktisch relevant, wie können sie als Risiko bewertet werden und wie muss ein Capability Contract strukturiert sein, damit er später durchsetzbar wird?

Für die Masterarbeit werden diese Fragen erweitert:

1. Wie kann ein MCP-Proxy den Contract mit geringer Latenz auf Tool-Aufrufe anwenden?
2. Welche im Hauptprojekt identifizierten runtime-abhängigen Angriffe lassen sich durch Proxy, Drift-Prüfung oder Intent-Check reduzieren?
3. Wann ist eine semantische Intent-Prüfung notwendig, und wie kann sie eingesetzt werden, ohne die Nutzbarkeit stark zu beeinträchtigen?

5 Methodischer Ansatz und Projektzuschnitt

Der methodische Kern ist der Vergleich zwischen *declared capabilities* und *observed capabilities*. Declared capabilities beschreiben, was ein Tool laut Beschreibung und Schema tun soll. Observed capabilities werden zweigeteilt betrachtet: als statische Preflight-Signale im Server-Code und als Live-Signale eines gestarteten oder instrumentiert ausgeführten MCP-Servers. Aus diesem Vergleich soll ein Capability Contract entstehen, der die zulässige Verhaltensgrenze eines Tools beschreibt.

Im Hauptprojekt wird dieser Ansatz zunächst als lokal nutzbarer Scanner mit begrenzter Live-Beobachtung untersucht. Der Prototyp soll lokale MCP-Server anbinden, ihre Tool-Schnittstellen erfassen, Tool-Beschreibungen und Schemas auswerten, beobachtbare Fähigkeiten im Code erkennen, ausgewählte Live-Signale eines gestarteten Servers erfassen und daraus Findings sowie einen maschinenlesbaren Contract erzeugen. Der Umfang bleibt bewusst begrenzt: Eine kleine Capability-Taxonomie mit Dateisystem, Netzwerk, Prozessausführung und Secrets reicht für den ersten Nachweis aus. Gleichzeitig soll der Scanner nicht nur Findings liefern, sondern die Grenze zwischen statischer Erkennbarkeit und Live-Erkennbarkeit markieren: Tool Poisoning in Metadaten, überprivilegierte Capabilities oder offensichtliche Code-Sinks sind eher statisch adressierbar; Rug-Pulls, indirekte Prompt Injection, Tool-Chaining-Missbrauch oder Intent-Mismatches benötigen dagegen Informationen aus dem laufenden MCP-Kanal. Verwandte Arbeiten wie *mcp-sec-audit*, Connor und VIPER-MCP zeigen, dass statische und instrumentierte Analysen für MCP-Server praktisch umsetzbar sind, adressieren aber jeweils andere Ziele: Sie liefern Risiko-Reports, Taint-Findings oder einmalige Malicious/Benign-Verdicts, nicht einen deklarationsbezogenen Capability Contract, der statische und live beobachtete Signale als Grundlage für spätere Enforcement-Entscheidungen zusammenführt [7–9].

Die Masterarbeit kann diesen Contract anschließend in eine Laufzeitperspektive überführen. Ein MCP-Proxy würde Tool-Aufrufe gegen den Contract prüfen, Drift oder Rug-Pulls gegenüber dem ursprünglich analysierten Zustand erkennen und bei sensitiven Operationen zusätzlich die Konsistenz zwischen Nutzerintention und Tool-Aufruf bewerten. Damit verschiebt sich der Fokus von der einmaligen Analyse auf die Frage, ob ein Contract im laufenden Agentenbetrieb eine praktikable Sicherheitsgrenze bilden kann. Der im Hauptprojekt entstehende Ausblick ist somit nicht nur ein Zusatz, sondern die Brücke zur Masterarbeit: Er begründet, welche Lücken des CLI-Scanners durch einen Proxy untersucht werden sollen.

6 Vorläufiger Arbeitsplan und offene Punkte

Das Arbeitsprogramm gliedert sich in mehrere Schritte. Zunächst werden Threat Model und relevante Literatur konsolidiert, um die betrachteten Angriffsklassen und die Abgrenzung zu bestehenden Scannern, Benchmarks und Laufzeit-Ansätzen zu schärfen. Darauf aufbauend werden eine begrenzte Capability-Taxonomie und ein erstes Contract-Schema definiert. Diese Vorarbeit legt fest, welche Fähigkeiten im Hauptprojekt betrachtet werden und welche Informationen ein Contract enthalten muss, um später von einem Proxy genutzt werden zu können.

Anschließend wird ein Prototyp entwickelt, der lokale MCP-Server anbindet, Tool-Metadaten und Schemas auswertet, statische Preflight-Signale analysiert und ausgewählte Live-Signale eines gestarteten Servers erfasst. Die Evaluation soll keine vollständige Vermessung des MCP-Ökosystems leisten, sondern reproduzierbar einschätzen, welche Divergenzklassen statisch erkennbar sind, welche Live-Beobachtung benötigen und welche erst durch einen späteren Laufzeit-Proxy adressierbar werden. Neben kontrollierten Testservern sollen, soweit verfügbar, bestehende Benchmarks und Corpora wie MCPTox, MCP Security Bench, MCP Pitfall Lab oder MCP-Universe einbezogen werden.

Bewusst offen bleibt, wie tief instrumentierte Live-Ausführung bereits im Hauptprojekt enthalten sein muss und welche Rolle KI-Modelle bei der Ableitung erwarteter Fähigkeiten spielen können, ohne selbst zur alleinigen Sicherheitsentscheidung zu werden. Zu klären ist außerdem, wie reproduzierbar Live-Signale erfasst und wie False Positives im späteren Laufzeitbetrieb bewertet werden sollen. Diese offenen Punkte bilden die Brücke zur Masterarbeit, in der der Contract in einem Laufzeit-Proxy durchgesetzt und gegen Drift, Rug-Pulls und Intent-Mismatches geprüft werden soll.

Vorläufiges Literaturverzeichnis

- [1] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv preprint arXiv:2503.23278, 2025. <https://arxiv.org/abs/2503.23278>.
- [2] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. arXiv preprint arXiv:2506.13538, 2025. <https://arxiv.org/abs/2506.13538>.

- [3] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv preprint arXiv:2508.14925, 2025. <https://arxiv.org/abs/2508.14925>.
- [4] Dongsun Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents. arXiv preprint arXiv:2510.15994, 2025. <https://arxiv.org/abs/2510.15994>.
- [5] Ruiqi Li, Zhiqiang Wang, Yunhao Yao, and Xiang-Yang Li. MCP-ITP: An Automated Framework for Implicit Tool Poisoning in MCP. arXiv preprint arXiv:2601.07395, 2026. <https://arxiv.org/abs/2601.07395>.
- [6] Zhihao Li, Boyang Ma, Xuelong Dai, Minghui Xu, Yue Zhang, Biwei Yan, and Kun Li. Don't believe everything you read: Understanding and Measuring MCP Behavior under Misleading Tool Descriptions. arXiv preprint arXiv:2602.03580, 2026. <https://arxiv.org/abs/2602.03580>.
- [7] Charoes Huang, Xin Huang, and Amin Milani Fard. Auditing MCP Servers for Over-Privileged Tool Capabilities. arXiv preprint arXiv:2603.21641, 2026. <https://arxiv.org/abs/2603.21641>.
- [8] Yiheng Huang, Zhijia Zhao, Bihuan Chen, Susheng Wu, Zhuotong Zhou, Yiheng Cao, Xin Hu, and Xin Peng. From Component Manipulation to System Compromise: Understanding and Detecting Malicious MCP Servers. arXiv preprint arXiv:2604.01905, 2026. <https://arxiv.org/abs/2604.01905>.
- [9] Pengyu Sun, Qishu Jin, Enhao Huang, Zifeng Kang, Xin Liu, Dakun Shen, and Song Li. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers. arXiv preprint arXiv:2605.21392, 2026. <https://arxiv.org/abs/2605.21392>.
- [10] Run Hao and Zhuoran Tan. MCP Pitfall Lab: Exposing Developer Pitfalls in MCP Tool Server Security under Multi-Vector Attacks. arXiv preprint arXiv:2604.21477, 2026. <https://arxiv.org/abs/2604.21477>.
- [11] Sonu Kumar, Anubhav Girdhar, Ritesh Patil, and Divyansh Tripathi. MCP Guardian: A Security-First Layer for Safeguarding MCP-Based AI System. arXiv preprint arXiv:2504.12757, 2025. <https://arxiv.org/abs/2504.12757>.
- [12] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. MiniScope: A Least-Privilege Framework for Authorizing Tool-Calling Agents. arXiv preprint arXiv:2512.11147, 2025. <https://arxiv.org/abs/2512.11147>.